SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

1.1. Purpose:

This System Analysis Document (SAD) presents the **technical architecture, system design decisions, and data flow structure** for the development of **ScrapSync: AI-Powered Industrial Symbiosis & Decision Intelligence Platform**.
The purpose of this document is to:

- Define how the system is structured technically (frontend, backend, AI, database)
- Explain how data flows across system components
- Describe how AI (Gemini 2.5 Flash) is integrated as a core service
- Serve as a **reference guide for developers, testers, and stakeholders**

Key elements covered in this document:

1. **Architecture**
   Cloud-based platform using a client-server model, with FastAPI as the backend engine and Supabase (PostgreSQL) as the persistent database, integrated with Google Gemini 2.5 Flash as the multimodal AI core.
2. **Data Flows**
   How a waste manifest image is uploaded by a manufacturer, processed through OCR and AI classification, matched against buyer demand nodes in the database, and returned as a smart contract recommendation.
3. **Model Process**
   End-to-end workflow of core features including the Synergy Engine (matchmaking), batch processing, predictive insights, and the ESG dashboard.
4. **Role of Reference**
   This document is a reference for all team members and evaluators to understand the high-level architecture, data models, and functional scope of the ScrapSync platform.

1.2. Background:

Industrial waste management in Malaysia and the broader Southeast Asian region remains highly manual and fragmented. Manufacturers generate significant volumes of recyclable byproducts — wood dust, metal scraps, plastic offcuts — yet lack an efficient mechanism to connect with buyers who require these materials. Existing methods rely on phone calls, physical brokers, and manual ledgers, leading to missed revenue opportunities and unnecessary landfill contributions.

ScrapSync addresses this gap by digitizing and automating the entire industrial waste trading pipeline. By leveraging multimodal AI (Google Gemini 2.5 Flash), ScrapSync bypasses traditional brittle OCR pipelines and directly interprets photographic evidence of waste, classifying it and cross-referencing it against a live buyer database in a single step.

© UMHackathon 2026

This platform is a greenfield solution with no prior version, designed from the ground up for the UMHackathon 2026 hackathon, with a vision to scale into a full B2B commodity trading marketplace.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Manufacturer	Lists available industrial waste, uploads waste documents/images, views match results and signs smart contracts.	Fast and accurate waste valuation, clear buyer recommendations, easy document upload, and revenue projections.
Buyer / Recycler	Browses available waste listings, views recommended suppliers, sends purchase requests, and negotiates pricing.	Access to verified waste suppliers, competitive pricing insights, and reliable material specifications.
Admin	Monitors platform activity, verifies company accounts, manages listings, and reviews platform statistics.	Clear oversight of all transactions, user verification tools, and platform health monitoring.
Development Team	Builds and maintains the platform including AI integrations, database schema, and frontend dashboard.	Clear API contracts, modular codebase, well-documented architecture, and reusable components.
QA Team	Validates system behavior across all user roles including edge cases for AI output parsing and database operations.	Defined functional scope, request/response formats, and testable acceptance criteria for each module.

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web Application (Desktop Browser, Enterprise Dashboard)
Architecture	Client-Server with AI Service Layer (Monolithic Backend with Modular Services)
Frontend	Streamlit — Single-Page Enterprise Dashboard with wide layout
Backend	FastAPI — Lightweight, high-speed Python RESTful API server
AI Engine	Google Gemini 2.5 Flash — Multimodal LLM for OCR, classification, matching, and insights
Database	Supabase (PostgreSQL) — Real-time relational database with Auth
Deployment	Local / Cloud (scalable to GCP or AWS)

ScrapSync is structured as a client-server system with a single primary client: a Streamlit-based web dashboard. The frontend communicates with a FastAPI backend, which orchestrates three core service flows: (1) document ingestion and AI processing, (2) buyer matching and smart contract generation, and (3) predictive analytics and ESG reporting. All persistent data is stored in a Supabase PostgreSQL database, which hosts demand_nodes (live buyer registry) and transactions (corporate ledger). Google Gemini 2.5 Flash serves as the centralized AI reasoning engine, replacing traditional brittle OCR pipelines with a single multimodal inference step.

2.1.2 LLM as Service Layer

Google Gemini 2.5 Flash is not a generic 'AI Module' — it is deeply integrated as a structured service layer across multiple modules. The following describes how Gemini is used at each stage:

- Prompt Construction: For each user action (e.g., waste upload, match request, forecast query), the FastAPI backend constructs a structured prompt that includes: (a) a task instruction, (b) the relevant database context (e.g., available demand_nodes from Supabase), and (c) the user-uploaded content (image or text). Prompts are templated and length-controlled to stay within token budgets.

© UMHackathon 2026

- Context Window Contents: The context window includes the system instruction, the uploaded document (as base64 image or extracted text), relevant demand_node records from Supabase (top 10 by relevance), and any historical transaction data needed for forecasting. Total context is capped at approximately 8,000 tokens per request.
- Response Parsing: Gemini is instructed to respond only in structured JSON format. The FastAPI backend receives the response, strips any markdown formatting, and parses it into a Python dictionary. Parsed fields are then passed to downstream components (e.g., contract renderer, dashboard updater, Supabase INSERT).
- Token Limits & Chunking: For batch processing (up to 50 documents), each document is processed in an independent Gemini call. Input images are compressed to under 1MB before encoding. Text inputs are truncated to 2,000 characters per document if they exceed the limit. Repeated queries (same waste type + same buyer pool) are cached in-memory for the session duration.

2.1.3 Dependency Diagram

The following describes all major API calls and component interactions in the ScrapSync system:

- Streamlit Dashboard. FastAPI Backend: All user actions (upload, match, batch process, forecast) trigger HTTP POST/GET calls to the FastAPI server running on localhost or a cloud endpoint.
- FastAPI Backend. Google Gemini 2.5 Flash API: The backend constructs and sends prompts via the Google Generative AI SDK. The Gemini API returns structured JSON responses which the backend parses and routes to the appropriate handler.
- FastAPI Backend. Supabase (PostgreSQL): The backend queries Supabase for demand_nodes (buyer registry) before sending context to Gemini, and performs INSERT operations to the transactions table after smart contracts are signed.
- Streamlit Dashboard. Supabase (Direct Read): The ESG dashboard and transaction ledger tabs perform direct reads from Supabase using the Supabase Python client for real-time display of signed contracts and impact metrics.
- Supabase Auth. User Sessions: User login and registration are handled by Supabase Auth. Role-based routing (Manufacturer / Buyer / Admin) is enforced at the Streamlit session level after authentication.

2.2.2. Sequence Diagram

The following describes the step-by-step interaction flow for the core Synergy Engine feature (Waste Upload → Match → Contract):

- Step 1 — Upload: Manufacturer drops a waste image (e.g., kayu.jpeg) into the Streamlit dashboard upload widget.
- Step 2 — Routing: Streamlit sends the image file to the FastAPI backend via an HTTP POST to /analyze endpoint.
- Step 3 — DB Query: FastAPI queries Supabase demand_nodes table to fetch all active buyers and their accepted material types.
- Step 4 — Prompt Build: FastAPI constructs a Gemini prompt including the image (base64-encoded), the buyer list, and task instructions: classify material, estimate quantity, find best buyer, calculate profit, draft contract.

© UMHackathon 2026

- Step 5 — AI Inference: Gemini 2.5 Flash processes the multimodal input and returns a structured JSON object with fields: material_type, quantity_kg, matched_buyer, projected_profit, contract_terms, reasoning.
- Step 6 — Parse & Respond: FastAPI parses the JSON, validates required fields, and returns the structured result to the Streamlit frontend.
- Step 7 — Display: Streamlit renders the match result: buyer name, projected profit (e.g., RM 24,000), logistics route, and the Digital Twin Smart Contract text.
- Step 8 — Sign Contract: Manufacturer clicks 'Sign & Execute Digital Twin Contract'. FastAPI performs an INSERT into the Supabase transactions table with all deal parameters.
- Step 9 — ESG Update: The transaction data is automatically reflected in the ESG dashboard (CO_2 avoided, landfill diverted metrics).

2.2. Technological Stack

Layer	Technology	Justification
Frontend	Streamlit (Python)	Rapid enterprise dashboard development. Wide-layout, single-page design suitable for B2B SaaS. No separate JavaScript needed, enabling fast prototyping.
Backend	FastAPI (Python)	High-speed asynchronous RESTful API framework. Ideal for orchestrating multiple service calls (Gemini, Supabase) with low latency. Auto-generates API docs via Swagger.
AI Engine	Google Gemini 2.5 Flash	Multimodal capability allows direct image interpretation without a separate OCR pipeline. Single-step waste classification, buyer matching, and contract drafting. Outputs structured JSON reliably.
Database	Supabase (PostgreSQL)	Managed PostgreSQL with built-in Auth, real-time subscriptions, and REST API. Hosts demand_nodes (buyer registry) and transactions (ledger). Zero infrastructure overhead.
AI SDK	Google Generative AI SDK (Python)	Official SDK for Gemini API integration. Handles authentication, request serialization, and streaming responses.

© UMHackathon 2026

Authentication	Supabase Auth	Handles user registration, login, and session management. Supports role-based access control for Manufacturer, Buyer, and Admin roles.

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD) — Level 0 (Context Diagram)

External Entities: Manufacturer, Buyer, Admin, Google Gemini API, Supabase Database.

Central Process: ScrapSync Platform (FastAPI Backend + Streamlit Frontend).

- Manufacturer → ScrapSync: Waste images, document uploads, manual waste entries, contract sign actions.
- Buyer → ScrapSync: Browse requests, purchase interest, negotiation offers.
- Admin → ScrapSync: Company verification actions, listing moderation commands.
- ScrapSync → Google Gemini API: Structured prompts with image data and buyer context.
- Google Gemini API → ScrapSync: Structured JSON with classification, match, profit, contract data.
- ScrapSync → Supabase: INSERT transactions, QUERY demand_nodes, UPDATE listing status.
- Supabase → ScrapSync: Buyer records, transaction history, user authentication tokens.
- ScrapSync → Manufacturer: Match results, profit projections, signed contract confirmation, ESG metrics.
- ScrapSync → Buyer: Recommended waste listings, supplier profiles, transaction status.

2.3.2. **Data Flow — Level 1 (Main Processes)**

- Process 1 — Waste Ingestion: Manufacturer uploads image → FastAPI routes to Gemini → Gemini extracts material type, quantity, condition → Parsed result stored temporarily → Passed to Process 2.
- Process 2 — Buyer Matching: Parsed waste data + demand_nodes from Supabase → FastAPI sends to Gemini for ranking → Gemini returns ranked buyer list with match scores and profit projections → Result passed to Process 3.
- Process 3 — Contract Generation & Ledger: Manufacturer confirms match → FastAPI generates Digital Twin Smart Contract via Gemini → Contract displayed to user → On sign, FastAPI INSERTs into Supabase transactions table → ESG metrics updated.
- Process 4 — Forecasting: FastAPI queries Supabase transaction history → Sends to Gemini with pricing trend context → Gemini returns sell-now vs. wait recommendation → Displayed on dashboard.

2.3.3. **Data Flow — Level 1 (Main Processes)**

The ScrapSync database is designed to Third Normal Form (3NF) to eliminate data redundancy and ensure referential integrity.

© UMHackathon 2026

Table	Key Fields	Purpose & Relationships
users	user_id (PK), email, password_hash, role, company_name, sector, location, created_at	Stores all registered users. role field (manufacturer/buyer/admin) drives routing. Referenced by waste_listings and transactions.
waste_listings	listing_id (PK), user_id (FK → users), material_type, quantity_kg, condition, location, availability_date, status, created_at	Each row is one waste listing by a manufacturer. Normalized: material classification is stored, not re-derived. References users table.
demand_nodes	node_id (PK), buyer_name, accepted_materials (array), location, price_per_kg, capacity_kg, active	Live buyer registry. One row per buyer per material type to maintain 3NF (no repeating groups). Used by Gemini context and matching algorithm.
transactions	txn_id (PK), listing_id (FK → waste_listings), buyer_id (FK → users), matched_buyer_name, quantity_kg, revenue_rm, co2_avoided_kg, landfill_diverted_kg, contract_text, signed_at	Immutable ledger of signed deals. Junction between waste_listings and demand_nodes buyers. Drives ESG dashboard calculations.
ai_analysis_log	log_id (PK), listing_id (FK), prompt_hash, gemini_response_json, tokens_used, latency_ms, created_at	Audit log of all Gemini API calls. Enables prompt caching (same prompt_hash = cached response). Supports cost monitoring.

© UMHackathon 2026

3. Functional Requirements & Scope

This section defines the core boundaries of the ScrapSync platform, focused on a high-impact MVP that demonstrates technical feasibility within the UMHackathon 2026 timeframe.

3.1. Minimum Viable Product:

The following 5 core features form the MVP to be built and demonstrated by the end of UMHackathon 2026:

NO	Feature	Description
1	Synergy Engine (Waste Upload & AI Matchmaking)	Manufacturer uploads a photo of a waste log (e.g., handwritten receipt, scanned invoice). Gemini 2.5 Flash reads the image, classifies the material (e.g., Wood Dust, 80kg), queries live demand_nodes from Supabase, identifies the highest-paying buyer (e.g., Aisha Fungi Farms), calculates projected profit (e.g., RM 24,000), and generates a Digital Twin Smart Contract. The signed deal is recorded directly into the Supabase transactions table.
2	Enterprise Batch Processing	Warehouse managers can upload up to 50 waste receipts/images simultaneously. The system processes each document independently via Gemini, aggregates all results, and outputs a 'Monthly Projected Profit' summary dashboard. Demonstrates platform scalability beyond single-document flow.
3	Predictive AI Insights & Decision Intelligence	ScrapSync analyzes historical transaction ledger data from Supabase to identify patterns. It alerts managers proactively (e.g., 'You consistently produce 500kg of sawdust by Friday; market rates are dropping next week — auto-book a truck now to lock in today's price'). Transforms the app from a reactive tool into a proactive revenue-generating asset.
4	ESG Dashboard & Corporate Ledger	Every signed smart contract automatically updates live ESG metrics: Carbon Avoided (kg CO2), Landfill Diverted (kg), Number of Successful Matches, and Total Revenue Generated. The immutable Supabase ledger enables large manufacturers to automate government sustainability reporting. Visualized as real-time graphs and metrics on the main dashboard.
5	Smart Buyer Matching with AI Explanation	Given a waste listing, the system ranks all available buyers from the demand_nodes registry using price per kg, location proximity, material compatibility, and demand volume. Gemini generates a human-

© UMHackathon 2026

		readable explanation of why the top buyer was recommended (e.g., 'Aisha Fungi Farms accepts wood waste up to 200kg, offers RM 300/kg, and is located 12km away — highest net profit after logistics cost').

3.2. Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation Strategy
Scalability	The system must handle batch uploads of up to 50 documents per session without timeout.	Each document is processed in an independent async Gemini API call. FastAPI uses async/await to prevent blocking. Future: add task queues (Celery/Redis) for >50 documents.
Reliability	AI match results must be deterministic for the same input. Failed Gemini calls must not corrupt the database.	Gemini temperature set to 0 for deterministic output. Supabase INSERTs are only triggered after successful contract sign action. Failed API calls return a graceful error state to the user.
Maintainability	Each functional module (waste listing, matching, forecasting, ESG) must be independently modifiable.	FastAPI routes are organized by module (/analyze, /match, /forecast, /admin). Streamlit pages are separated into tabs. Supabase schema changes are managed via migrations.
Token Latency	Gemini API calls must return within 5 seconds under normal load. Users must see a loading indicator during inference.	Gemini 2.5 Flash is selected for its speed over Gemini Pro. Streamlit st.spinner() displays a loading state. Async FastAPI calls prevent UI blocking. Timeout set to 10s with graceful fallback message.
Cost Efficiency	Average Gemini token cost per user session must stay within hackathon budget constraints.	Prompts are templated with fixed-length structure. Images are compressed to under 1MB before base64 encoding. Demand_nodes context is limited to top 10 buyers (not full table). Repeated queries within a

© UMHackathon 2026

		session are cached using prompt_hash in ai_analysis_log.
Security	User data and company information must be protected. Only authenticated users may access their own listings.	Supabase Auth handles JWT-based authentication. Row-Level Security (RLS) policies on Supabase ensure users can only read/write their own records. No API keys are exposed to the frontend.

3.3. Out of Scope / Future Enhancements

The following features represent the 'Grand Vision' for ScrapSync but are not included in the UMHackathon 2026 MVP:

- In-App Chat / Negotiation Messenger: Real-time messaging between manufacturer and buyer during deal negotiation. Planned as Module 6 (Transaction & Negotiation).
- Mobile Application: Native iOS and Android apps for on-site waste logging by warehouse staff.
- Automated Logistics Booking: Integration with third-party logistics APIs (e.g., Lalamove, DHL) to automatically book trucks upon contract signing.
- Blockchain Smart Contracts: Migration from Digital Twin contracts to actual on-chain smart contracts for immutable and legally enforceable deal records.
- Regulatory Compliance Automation: Automated generation of DOE (Department of Environment Malaysia) waste disposal reports and sustainability certificates.
- Marketplace Bidding System: Open bidding where multiple buyers compete for a waste listing, driving up seller revenue.
- Multi-language Support: Bahasa Malaysia and Mandarin interfaces for wider adoption across Malaysian manufacturers.

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. **Grayscale Rollout & A/B Testing**: Do highlight team's strategy of deployment [e.g. new feature of user-preferred food menu are released to 5% of real users initially, monitoring and then gradually increased until 100% if no issues are detected. After it passes, A/B testing is conducted to determine which variants of two strategies perform better.]

4.1.2. **Emergency Rollback (ER) & Golden Release**: Do mentioned the strategies for Emergencies. [e.g. In terms of recovery & traffic management for YFood system, if there are anomalies, ER is triggered. The automated pipeline does deploy the Golden Version of the application.]

4.1.3. **Priority Matrix:**

4.1.3.1. Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project. [e.g. P1 Critical: If payment failure error is more than 1%, emergency rollback to Golden release.]

4.2. Monitoring:

4.2.1. Show the planning for your monitoring plan of the system and how it will trigger if any damage is detected of the health of the system.

4.3. Assumptions: State key operational & environmental conditions that you assumed to be valid while development of system and deployment. [e.g.

a. Users have stable internet connections on their devices during the placement of orders
b. Restaurants have their own devices to access the web-dashboard for monitoring of orders]

4.4. External Dependencies: Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks
Z Ai GLM API	It should be the core LLM reasoning engine - that will handle the prompt	High- in case of GLM API rate limitation - core product will fail. So,

© UMHackathon 2026

	inference, multi-step reasoning and generating structured output	implement a mitigation strategy [e.g. graceful degradation state from users]
Google Maps API	Tracking rider location and optimize the route	Medium level- if real time tracking is being disabled.

© UMHackathon 2026

5. Project Management & Team Contributions

The UMHackathon 2026 preliminary round spans approximately 10 days, structured as a two-week sprint. The team has prioritized the Synergy Engine and ESG Dashboard as the highest-impact deliverables for the live demonstration.

5.1. Project Timeline

Phase	Days	Tasks & Deliverables
Project Planning & Setup	Day 1–2	Finalize system architecture, set up Supabase schema (demand_nodes, transactions, users, waste_listings), configure FastAPI project structure, connect Gemini API, seed demand_nodes with test buyer data, establish GitHub repository and branching strategy.
Core AI Engine Development	Day 3–4	Implement /analyze FastAPI endpoint, build Gemini prompt templates for waste classification and buyer matching, integrate Google Generative AI SDK, build JSON response parser, implement error handling and graceful degradation for API failures.
Frontend Dashboard Development	Day 5–6	Build Streamlit dashboard with wide layout: Upload tab (Synergy Engine), Batch Processing tab, ESG Dashboard tab (CO2/landfill graphs), Transaction Ledger tab, Supply Chain Map tab. Integrate with FastAPI backend via HTTP calls.
Advanced Features & Forecasting	Day 7–8	Implement Predictive AI Insights module (historical data analysis, sell-now vs. wait recommendation), complete Digital Twin Smart Contract generation and Supabase INSERT flow, implement Admin module (basic user verification and platform stats), add match score ranking to buyer display.

© UMHackathon 2026

Testing, Refinement & Documentation	Day 9–10	End-to-end testing of all 5 MVP features with realistic test data, fix identified bugs (e.g., manual waste entry form, edit/delete listing), performance optimization (prompt compression, image resizing), complete System Analysis Documentation (SAD), prepare live demo script and click-through flow.

5.2. Team Members Role

Team Member	Role	Responsibilities
MUHAMMAD ADEEB 'IRFAAN BIN ABDUL HANIM	Project Director	Lead overall project planning and execution, coordinate team members, monitor progress and timeline, make key decisions, manage risks and issues, ensure all components are integrated smoothly, and oversee final delivery of the projec
MUHAMMAD YASRIN BIN ROSALAN	AI Integration & Prompt Engineering	Design and optimize all Gemini prompt templates, implement FastAPI /analyze and /forecast endpoints, handle JSON response parsing, manage Gemini API integration and error handling, oversee ai_analysis_log schema.
MUHAMMAD ARIF AIMAN BIN KARIM	FastAPI Backend & Database	Design and implement Supabase schema (all 5 tables), configure Row-Level Security policies, implement /match and /admin endpoints, handle Supabase query optimization, manage user authentication flow.
AHMAD SUHAIB BIN ZULFIKRI	Streamlit Dashboard & UX	Build all Streamlit tabs and UI components, implement file upload widgets, display match results and ESG graphs, integrate frontend with FastAPI endpoints, ensure responsive wide-layout design quality.
DANISH BIN ISMAIL DENNIS	Data Seeding & ESG Metrics	Populate demand_nodes with realistic buyer data, design

© UMHackathon 2026

		ESG metric calculations (CO2 avoided formula, landfill diversion formula), validate AI output accuracy against test cases, prepare demo data and demo script.

5.3. Recommendations

The following technical recommendations are proposed to improve ScrapSync's scalability, performance, and reliability beyond the hackathon MVP:

- Redis Cache for Demand Nodes: The demand_nodes table is queried for every Gemini call. Implement a Redis in-memory cache with a 15-minute TTL to reduce Supabase read load during batch processing sessions.
- Celery + Redis Task Queue for Batch Processing: Replace synchronous batch processing with an async Celery task queue. This allows 50+ documents to be processed in parallel workers, reducing total batch time from O(n) sequential to near-O(1) parallel.
- Prompt Caching via ai_analysis_log: Before sending any Gemini request, compute a hash of the prompt template + image content. Check ai_analysis_log for an existing entry with the same hash. If found (and less than 24 hours old), return the cached response. This directly reduces API costs for repeated waste types.
- Image Preprocessing Pipeline: Before sending images to Gemini, apply automatic rotation correction, contrast enhancement, and compression to under 500KB. This improves OCR accuracy for low-quality handwritten documents and reduces token count.
- Supabase Row-Level Security (RLS): Enable RLS on waste_listings and transactions tables to ensure manufacturers can only view and modify their own records, and buyers can only view listings relevant to their accepted material types.
- Structured Logging & Monitoring: Integrate structured logging (e.g., Python logging with JSON formatter) for all FastAPI requests and Gemini API calls. Export logs to a monitoring dashboard (e.g., Grafana + Loki) for production deployment.
- Contract Digital Signing via DocuSign / PDF Generation: Replace the current in-app contract display with a downloadable PDF contract generated using a Python PDF library (e.g., ReportLab). In future, integrate DocuSign API for legally binding electronic signatures.

© UMHackathon 2026